jigsaw-agile-community-rules

# 4th place writeup - Instruct LLM is all you need

| | |
|---|---|
| **Rank** | 4 |
| **Team** | Dream Team |
| **Author** | Ruslan Berdichevsky |
| **Topic ID** | 613473 |
| **Version** | Original |

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/613473
Retrieved with Kaggle CLI on 2026-10-21.

# Kaggle Jigsaw (4th place) — Technical Write-up – Instruct LLM is all you need

## Acknowledgments

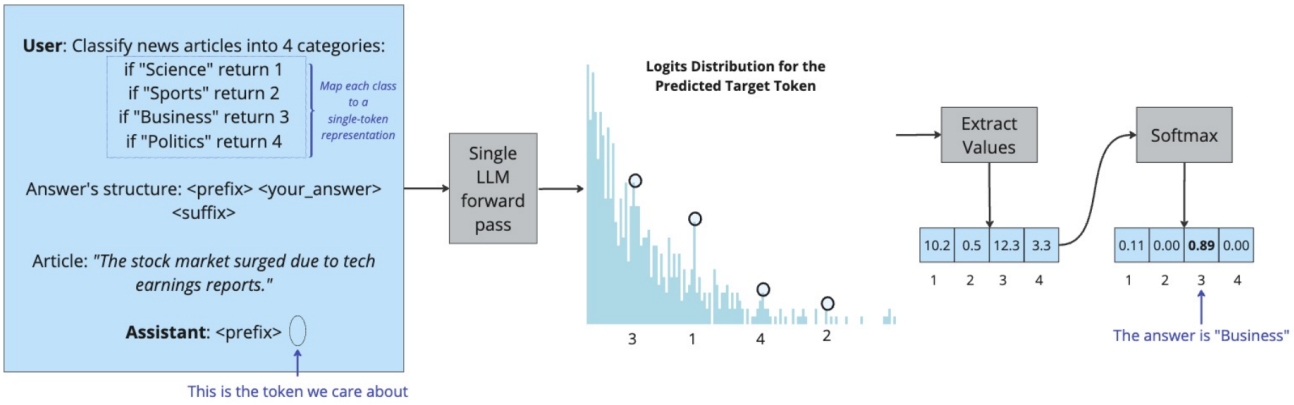
Many thanks to the organizers for a well-run competition. As someone new to this community, it was a great learning experience. This challenge also showcased how instruction-tuned LLMs, combined with tunable data, can deliver strong classification performance—especially in settings where **SALSA** (Single-pass Autoregressive LLM Structured Classification; paper) [1] can shine. **SALSA is my recent work—a research paper I conducted over the past year.**

## Overview

My submission is based on **SALSA** (Single-pass Autoregressive LLM Structured Classification). With a prompt template and tuning aligned to the task, an instruction-tuned LLM can achieve near-SOTA performance. In SALSA, the model's next-token **logits** over a tiny, fixed answer vocabulary (e.g., `"0"`, `"1"`) are treated as a **Bayesian estimator** of the label. In general, the larger and better aligned the instruction model, the stronger the SALSA signal—because training starts from a stronger zero-shot baseline. To squeeze out a bit more AUC, I introduced **Generalized Cross-Entropy (Tsallis)** and a **pseudo-labeling** stage.

### SALSA in one paragraph

- **Single pass:** Ask a tightly scoped question and read the probability of the answer token directly from the next-token distribution (no multi-step decoding).
- **Structured label space:** Constrain answers to a tiny vocabulary so logits map cleanly to class probabilities.
- **Bayesian view:** Normalized logits serve as likelihoods; you can use them as probabilities, combine across prompts/rules, or blend across models. This also enables effective use of **soft labels** and **pseudo-labeling**.



# Data processing

- **Few-shots as labels:** Every record included **four** few-shot exemplars—**2 positive** and **2 negative**. I used these as labels alongside the publicly provided labels.
- **Subreddit field removed:** I removed the **subreddit** field because it was misleading—the few-shot examples were unrelated to the subreddit field in the public data.
- **Duplicates & inconsistencies:** The training set contained redundant texts and occasional disagreements. I **collapsed duplicates** to create a unique dataset and computed the **rule-violation ratio** per unique text; during training, this ratio served as a **soft target** (ground-truth probability derived from the positive/total labels for the same comment–rule pair).

# Base SALSA training

**Compute constraints.** Training ~32B-parameter (or larger) models within Kaggle's time budget was a significant technical challenge, so I did not pursue them. I used **AWQ** base models because, in my experience, AWQ preserves model capability per bit better than GPTQ or simple weight clipping. **Backbones tried.** Qwen-3 **4B** and Qwen-3 **14B**.

**Robust loss.** To mitigate label noise, I replaced cross-entropy with **Generalized Cross-Entropy (Tsallis)** and set **q = 0.955** based on local CV and public LB sampling. This improved the public score, while the private score changed only marginally—suggesting a modest effect.

**Seed sensitivity.** I observed ~**0.002 AUC** variation from dataloader seed changes. Gains below that threshold (including from GCE) are hard to claim as statistically significant under these constraints.

## Single-stage results

| Model | Private | Public |
|---|---|---|
| Qwen-3 4B + **GCE** | 0.91874 | 0.92482 |
| Qwen-3 14B + **GCE** | 0.92313 | 0.92909 |
| Qwen-3 14B + **CE** | 0.92309 | 0.92803 |

# Prompting & inference

To keep the large model within budget, I used a **short template** and placed the **comment at the end** to maximize **prefix KV-cache** reuse. At inference, I **batched by rule** so the rule text also stayed cached.

**Prompt Template**

```
<|im_start|>user

Does the comment violate the rule? Answer  the :
<ANSWER>
Where the number  one of:
 - No
 - Yes
```

```
The rule:
<RULE>
{rule}
</RULE>

The Comment:
<COMMENT>
{body}
</COMMENT>
<|im_end|>
<|im_start|>assistant
<think>

</think>
<ANSWER>
```

For the smaller Qwen-3 4B, moving the instruction to the **end** produced a very small improvement, below statistical significance in this setup.

# Two-stage training with pseudo-labels

To push AUC further within the same compute envelope:

1. **Stage 1 (small model).** Train a smaller model and run inference on the target test domain.
2. **Uncertainty sampling.** Select **~8,000** most-uncertain (most informative) test samples as soft **pseudo-labels** for Stage 2.
    - These pseudo-labels **do not add new external knowledge**; instead, they **anchor relative scores** among test samples, letting the instruction model's pretrained knowledge be used more effectively.
3. **Stage 2 (bigger model).** Finetune **Qwen-3 14B** on the original data **plus** the pseudo-labeled set.
4. **Blending.** Blend Stage-1 and Stage-2 predictions with weights **0.8 : 0.2**.

## Final results

| System / Ablation | Private | Public |
|---|---|---|
| **Winning solution (blend 0.8 / 0.2)** | **0.9286** | **0.93382** |
| No blend (Stage-2 only) | 0.9264 | 0.93111 |
| Qwen-3 4B (single stage) | 0.91874 | 0.92482 |
| Qwen-3 14B (single stage) | 0.92313 | 0.92909 |
| Two stages with Qwen-3 4B | 0.92239 | 0.92642 |
| Qwen-3 4B (single stage) and Qwen-3 14B (single stage) blend | 0.92469 | 0.92921 |

**Practical note.** The two-stage pipeline increased engineering complexity. The ~**0.004 AUC** gain was decisive for rank here, but whether it's "worth it" will depend on a team's time/compute budget.

**Reference**

[1] **SALSA: Single-pass Autoregressive LLM Structured Classification.**